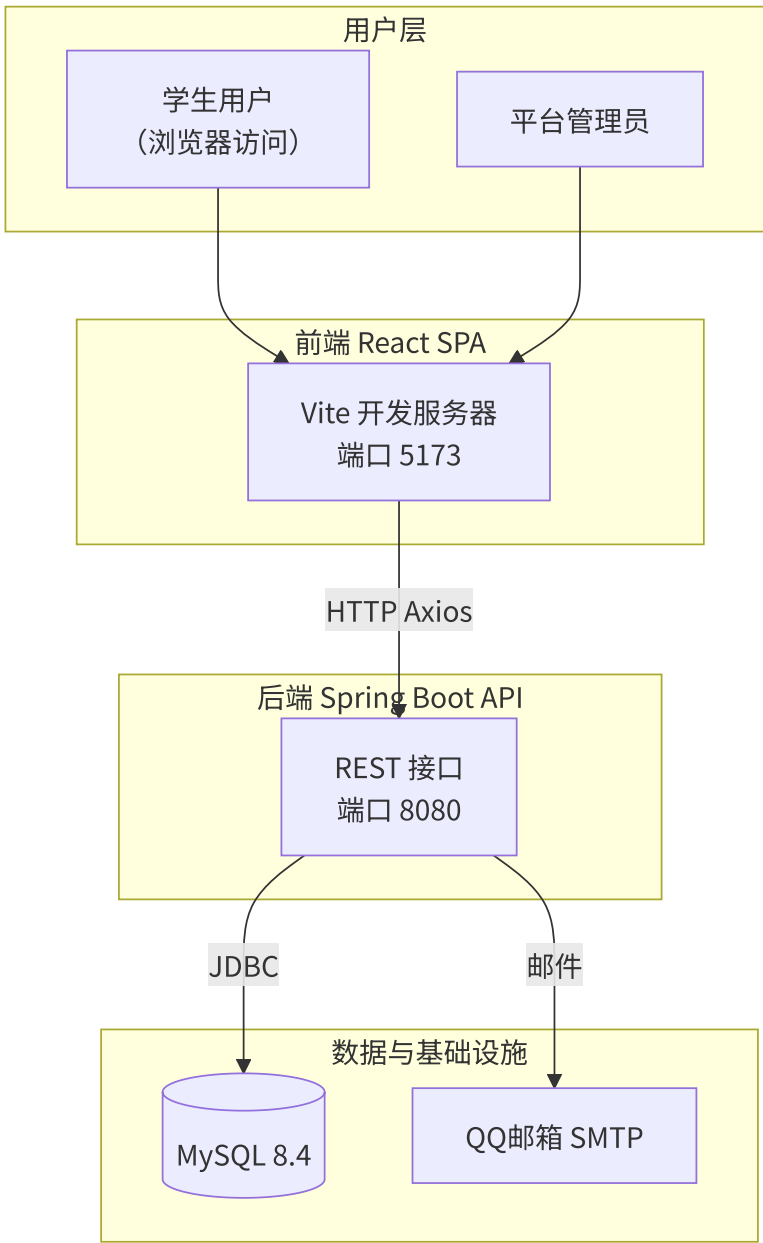


WHU Circle 需求建模文档

一、用户需求说明

1.1 项目概述

WHU Circle 是面向武汉大学学生的轻量级社交媒体原型系统，采用前后端分离的单页 Web 应用架构。系统以动态分享、兴趣频道、私信沟通、个人主页和隐私控制为核心，支持校内邮箱注册、图文笔记发布、关注与社交圈、频道讨论、即时通讯等校园社交场景。

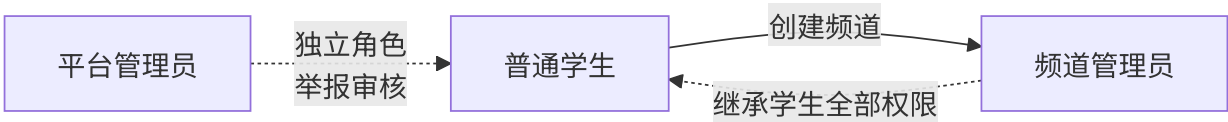


项目已实现后端 39 条 API 路径、50 个 HTTP 操作，覆盖认证、笔记、社交、频道、聊天、通知、举报等模块。前端完成 7 个核心页面：首页广场、社交圈、频道页、聊天页、个人主页、收藏页、设置页。

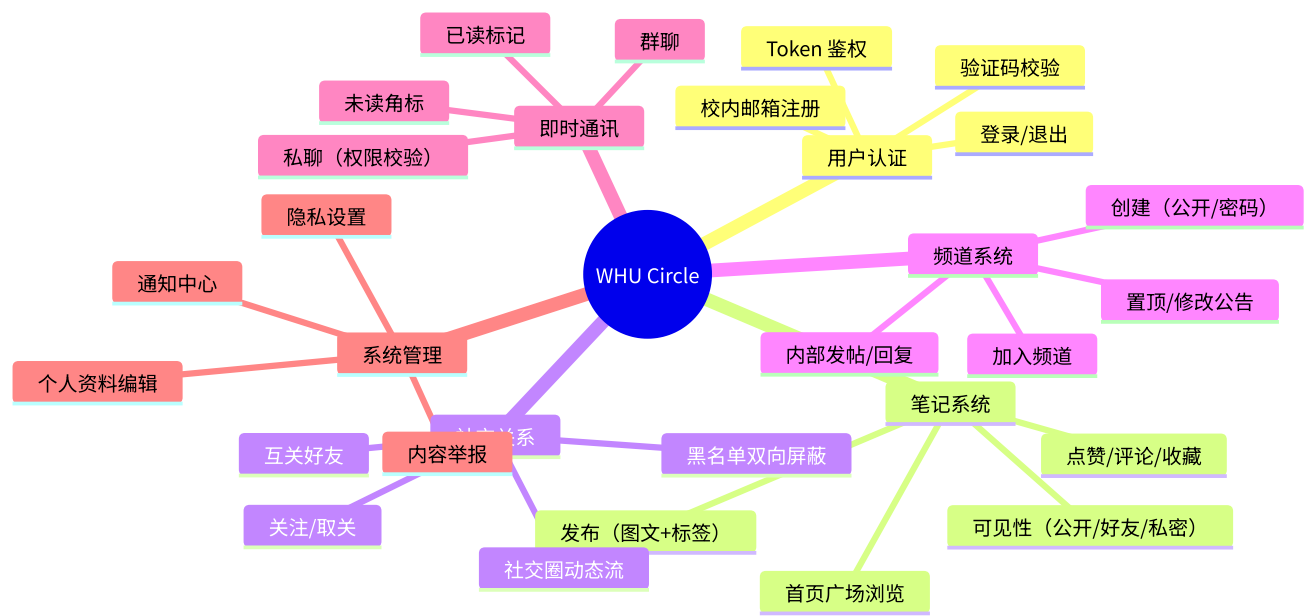
1.2 用户角色

系统定义三类用户角色，分别对应不同权限范围：

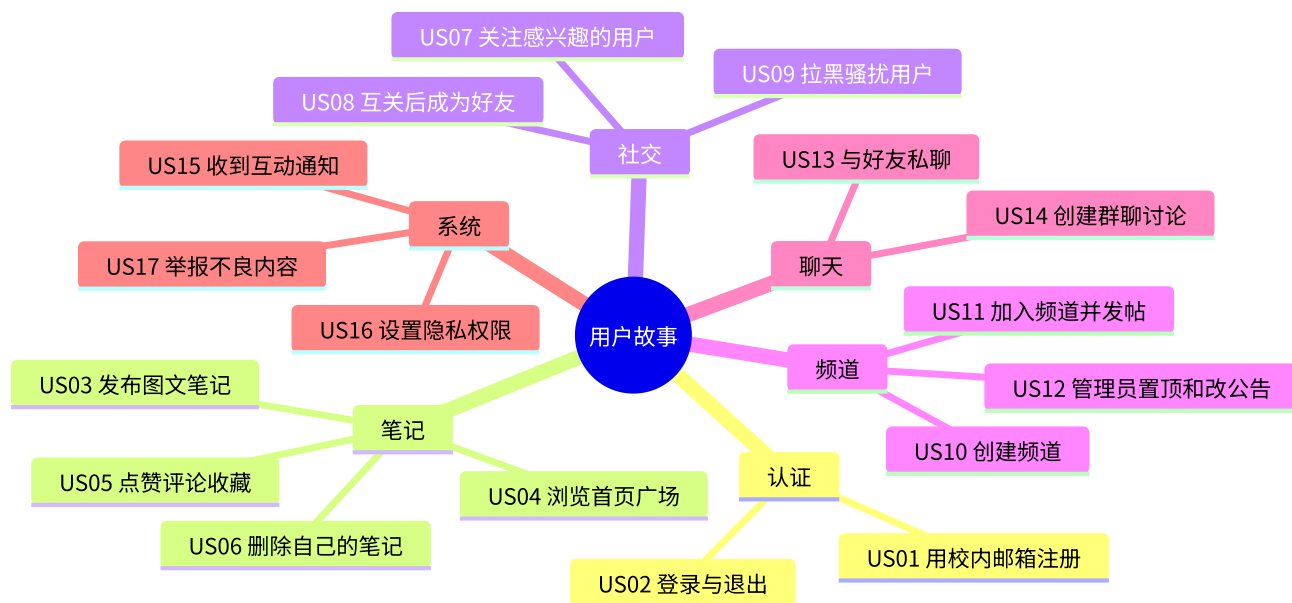
角色	权限说明	来源依据
普通学生	注册登录、发布/删除笔记、评论点赞收藏、关注取关拉黑、加入/创建频道、私信群聊、隐私设置、举报	Controller 层鉴权逻辑
频道管理员	继承学生所有权限；额外：修改公告、置顶/取消置顶帖子、管理频道成员	ChannelService.updateAnnouncement / setPinned
平台管理员	审核举报（RESOLVED/REJECTED），不可越权查看私密内容	ReportController + ReportService



1.3 功能模块总览



1.4 用户故事



1.5 核心业务需求

用户认证：仅 @whu.edu.cn 邮箱可注册。验证码 5 分钟有效、60 秒重发冷却、最多连续输入 5 次错误。密码使用 BCrypt 哈希存储。登录后服务端签发 Bearer Token，客户端在请求头携带。除 /auth/* 和 /health 外，所有 API 均需 Token 鉴权，无效 Token 返回 40100。

笔记系统：用户可发布笔记，包含标题（≤100 字）、正文（必填）、图片、标签。可见性分 PUBLIC（公开）、FRIENDS（好友可见）、PRIVATE（仅自己）。首页广场展示公开笔记，支持关键词搜索和标签筛选，分页返回（每页 20 条）。笔记详情按可见性校验：好友可见需互关关系（FRIEND），私密仅作者本人。点赞和评论支持切换状态与计数更新。评论 ≤1000 字，被拉黑用户不可评论。仅作者可删除笔记和评论。收藏笔记可在"我的收藏"中查看。

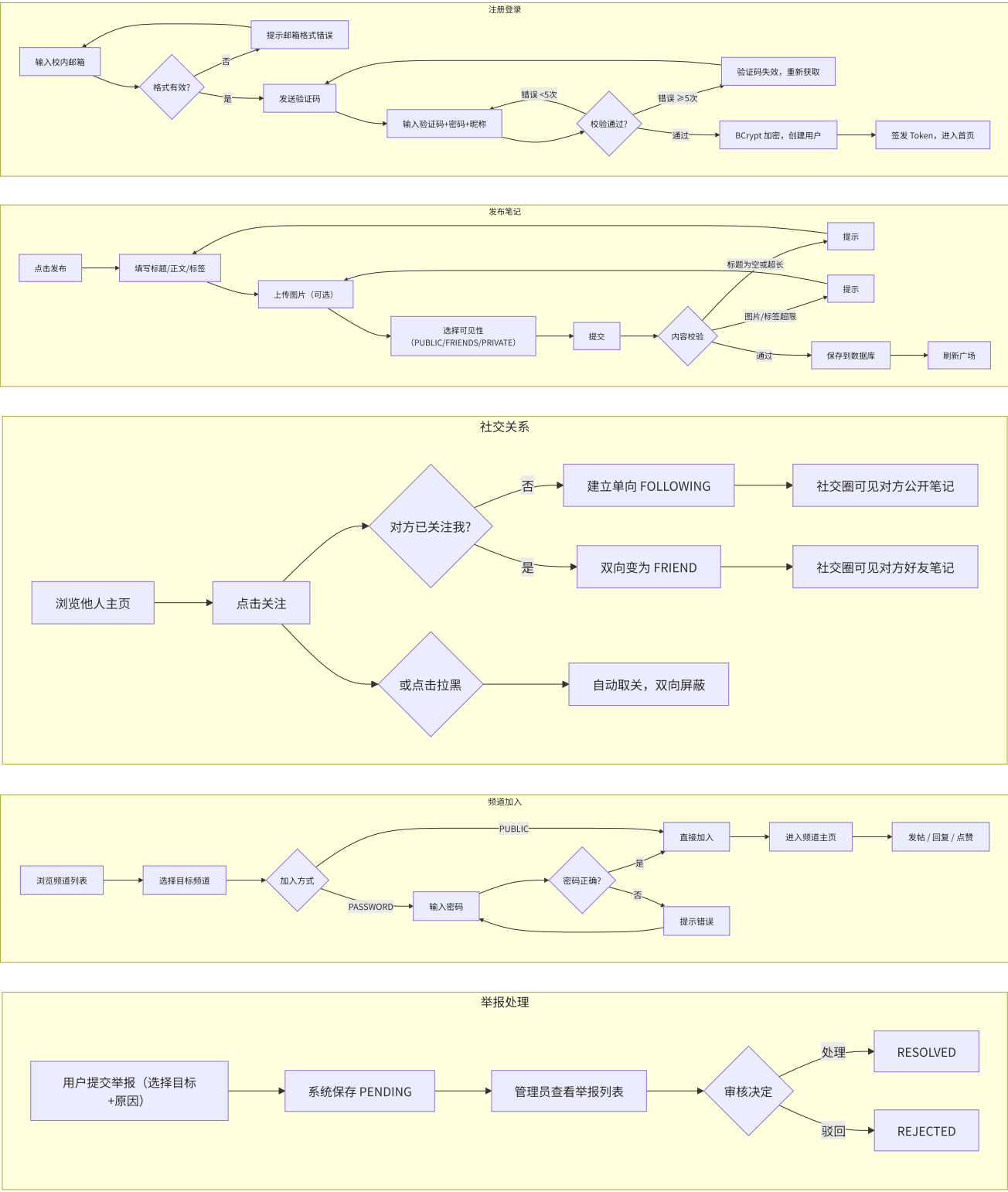
社交关系：关注为单向操作。若对方已关注自己，关系自动升级为 FRIEND（互关好友）。取关后好友关系自动降级。社交圈（/feed/social）仅展示已关注用户的笔记，好友可见内容需互关才显示。拉黑时自动取关，双向屏蔽主页访问。关系状态枚举：NONE → FOLLOWING → FOLLOWER → FRIEND → BLOCKED。

频道系统：支持 PUBLIC（公开加入）和 PASSWORD（密码验证）两种加入方式。创建者自动成为管理员和首位成员。仅成员可发帖、回复（≤1000 字）和点赞。未加入频道时仅预览最近 5 条帖子。仅管理员可修改频道公告和置顶/取消置顶帖子。

即时通讯：支持 PRIVATE 私聊和 GROUP 群聊。创建私聊会话时需校验黑名单关系和对方私信权限（EVERYONE / FRIENDS_ONLY / NONE）。消息 ≤2000 字，当前采用 HTTP 请求模式（非 WebSocket）。支持消息已读标记。会话列表按最后消息时间倒序，显示未读消息数。

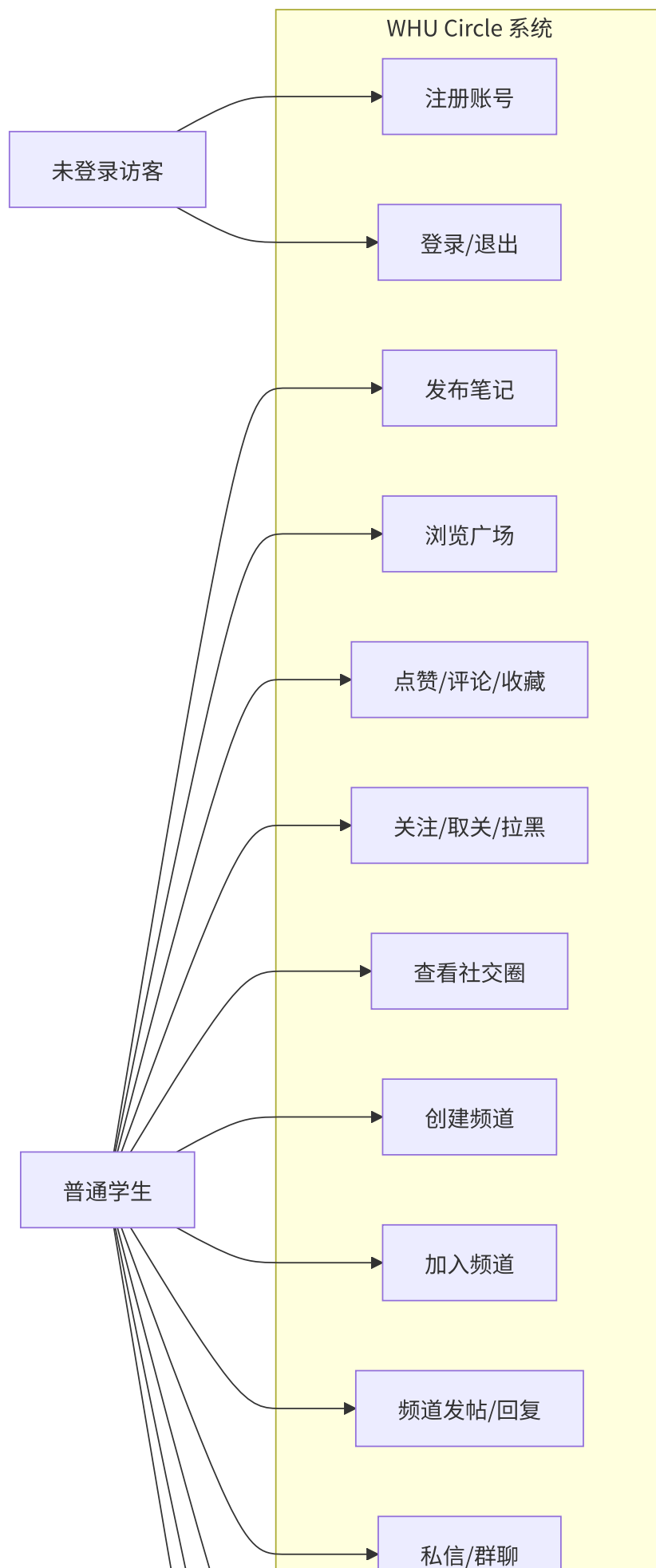
通知与安全：笔记被点赞、评论、收藏以及频道帖子被回复时，自动为内容作者生成未读通知。支持通知分页列表、单条已读、全部已读、未读数角标。隐私设置三项：默认笔记可见性、默认频道加入方式、私信权限。举报目标类型：NOTE（笔记）、CHANNEL_POST（频道帖子）、MESSAGE（消息）、USER（用户）。举报状态流转：PENDING → RESOLVED / REJECTED。

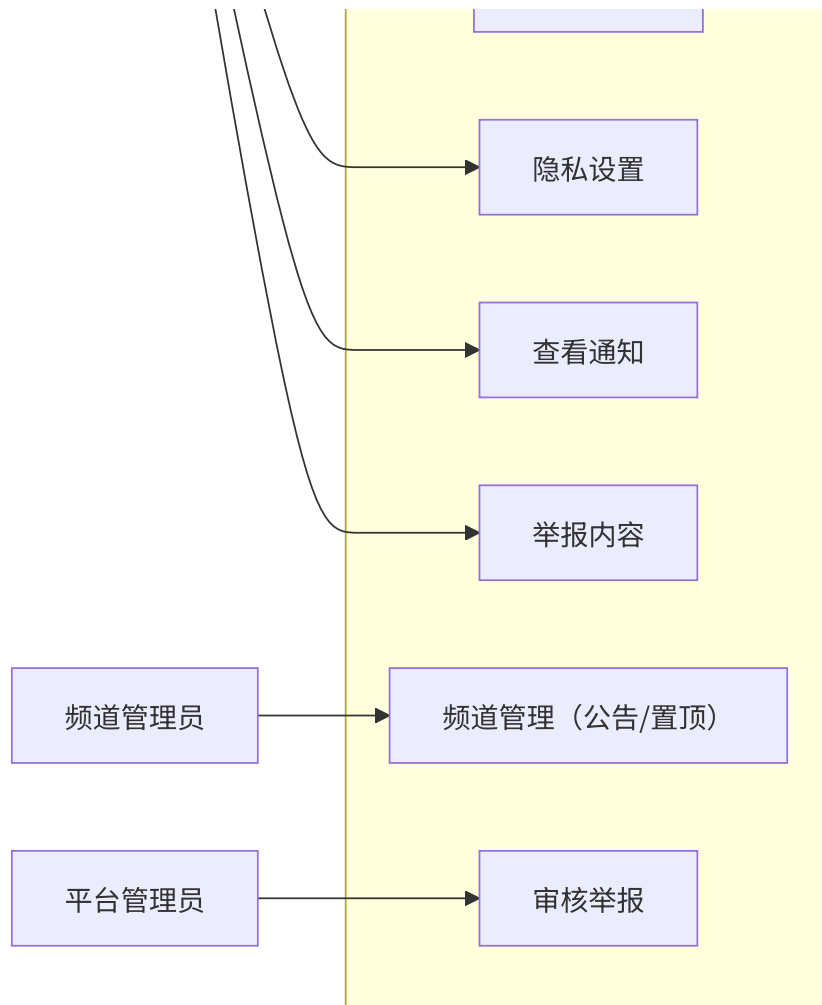
1.6 业务流程图



二、需求分析建模

2.1 用例图

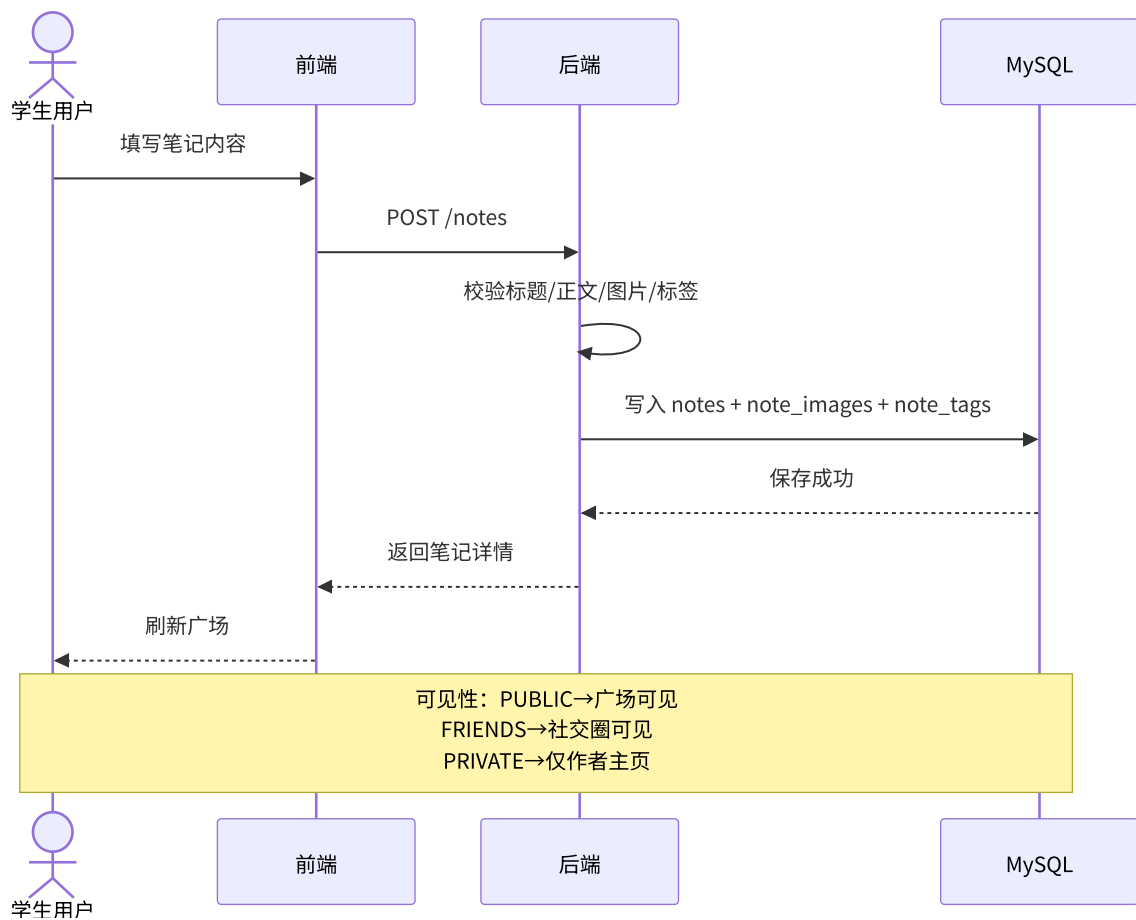
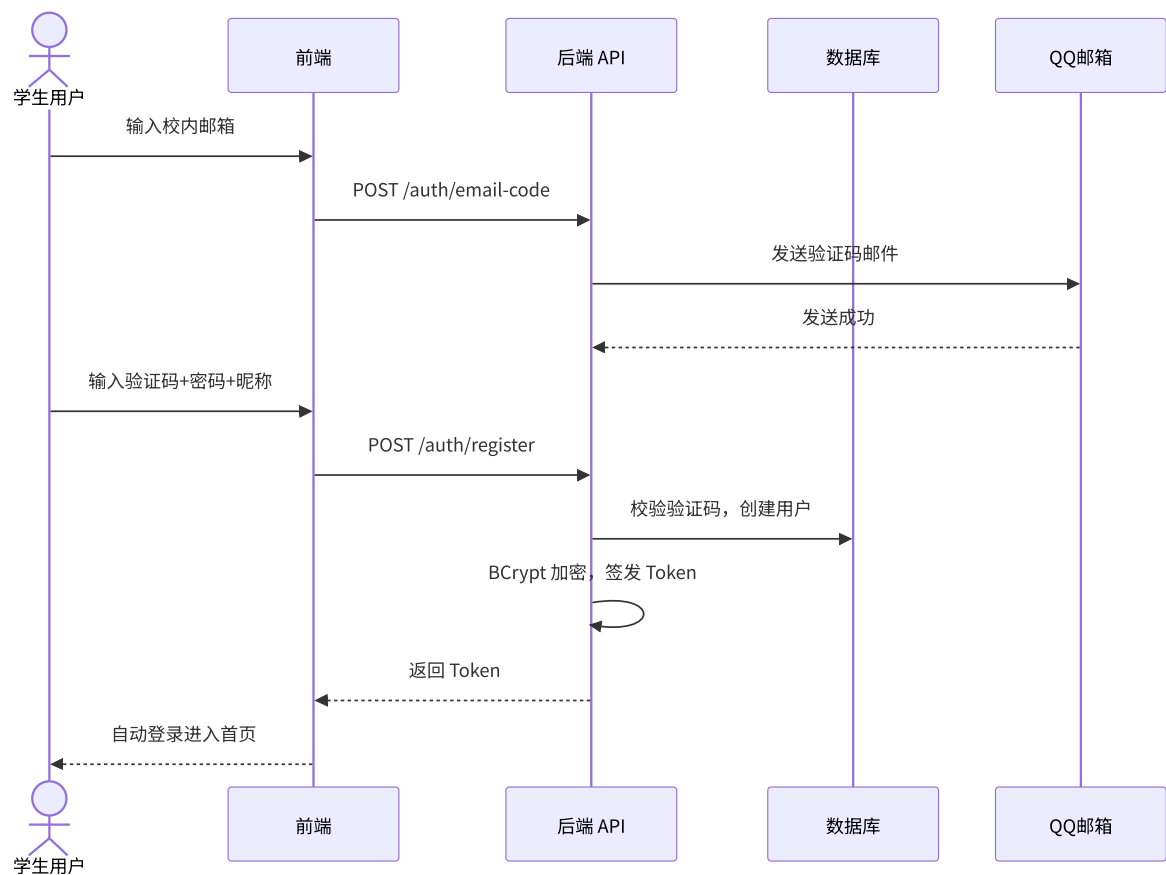


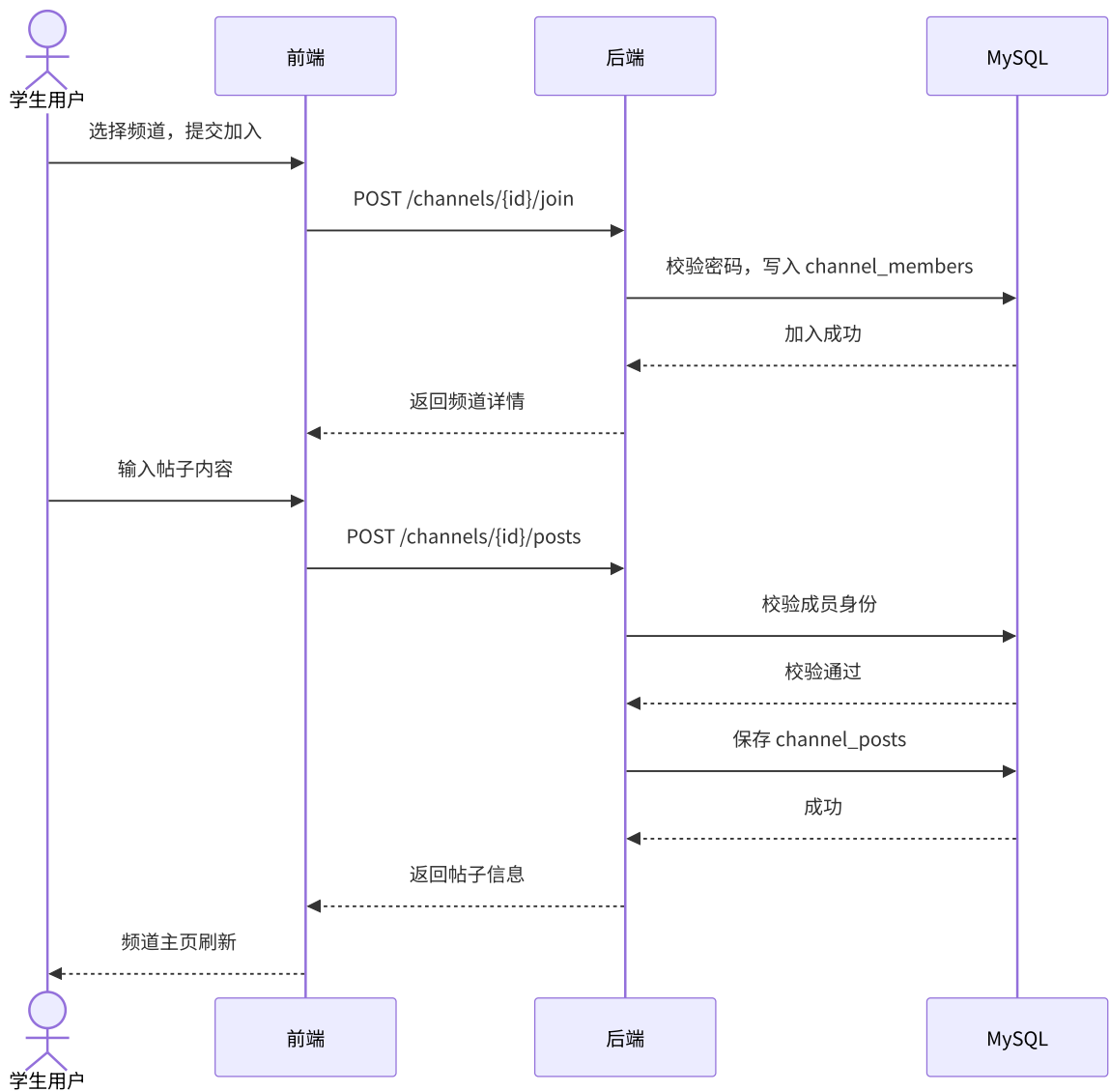
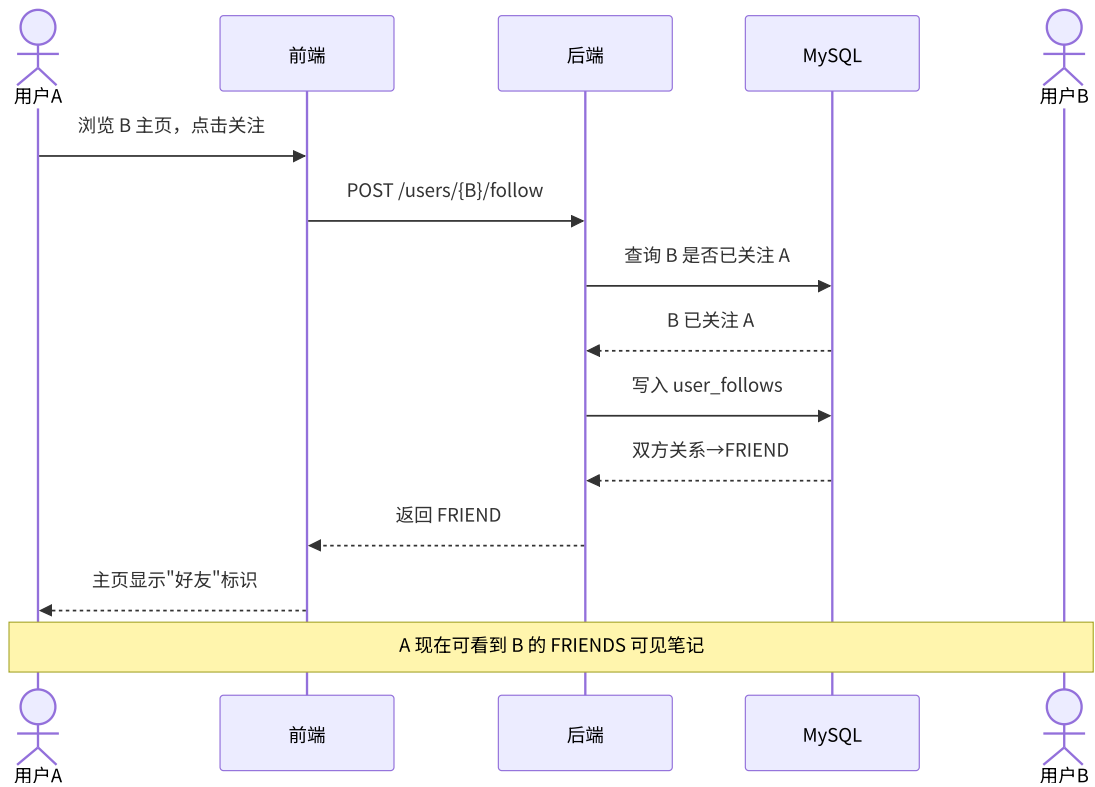


2.2 用例规约

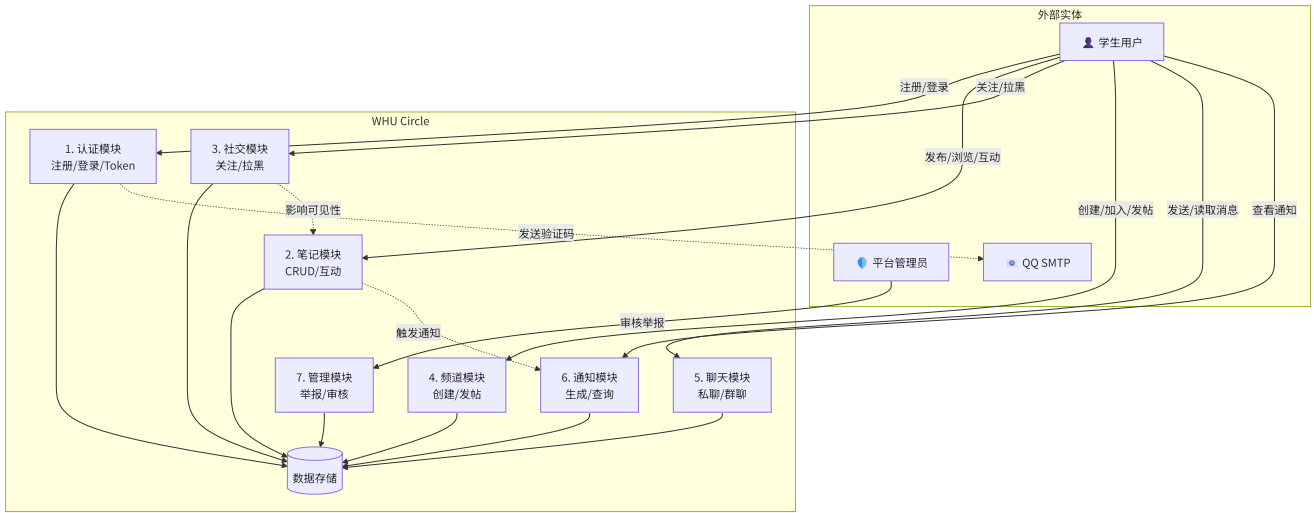
用例	参与者	前置条件	后置条件	基本流程	异常/备选
UC01 注册	未注册学生	拥有 whu.edu.cn 邮箱	用户创建成功, 自动登录	1. 输邮箱 2. 获验证码 3. 输验证码+密码 4. 校验 5. 签发 Token	邮箱已注册→409; 验证码错≤5次可重试, ≥5次失效
UC03 发布笔记	已登录学生	Token 有效	笔记保存, 出现在相关页面	1. 填标题/正文 2. 选可见性 3. 传图片 4. 提交 5. 校验 6. 保存	图片超限/标题为空→提示; 未登录→跳转
UC06 关注	已登录学生	进入他人主页	关系状态更新	1. 浏览主页 2. 点关注 3. 建立 FOLLOWING; 若互关→FRIEND	拉黑→自动取关并屏蔽
UC09 加入频道	已登录学生	Token 有效	成为频道成员	1. 选频道 2. 公开直接加入/密码校验后加入 3. 进入频道	密码错误→提示重新输入
UC12 私信	已登录学生	Token 有效, 未被对方拉黑	消息保存, 对方收到未读	1. 创建/选会话 2. 输消息 3. 校验权限 4. 保存消息	被拉黑→403; 私信权限 NONE→拒绝

2.3 时序图

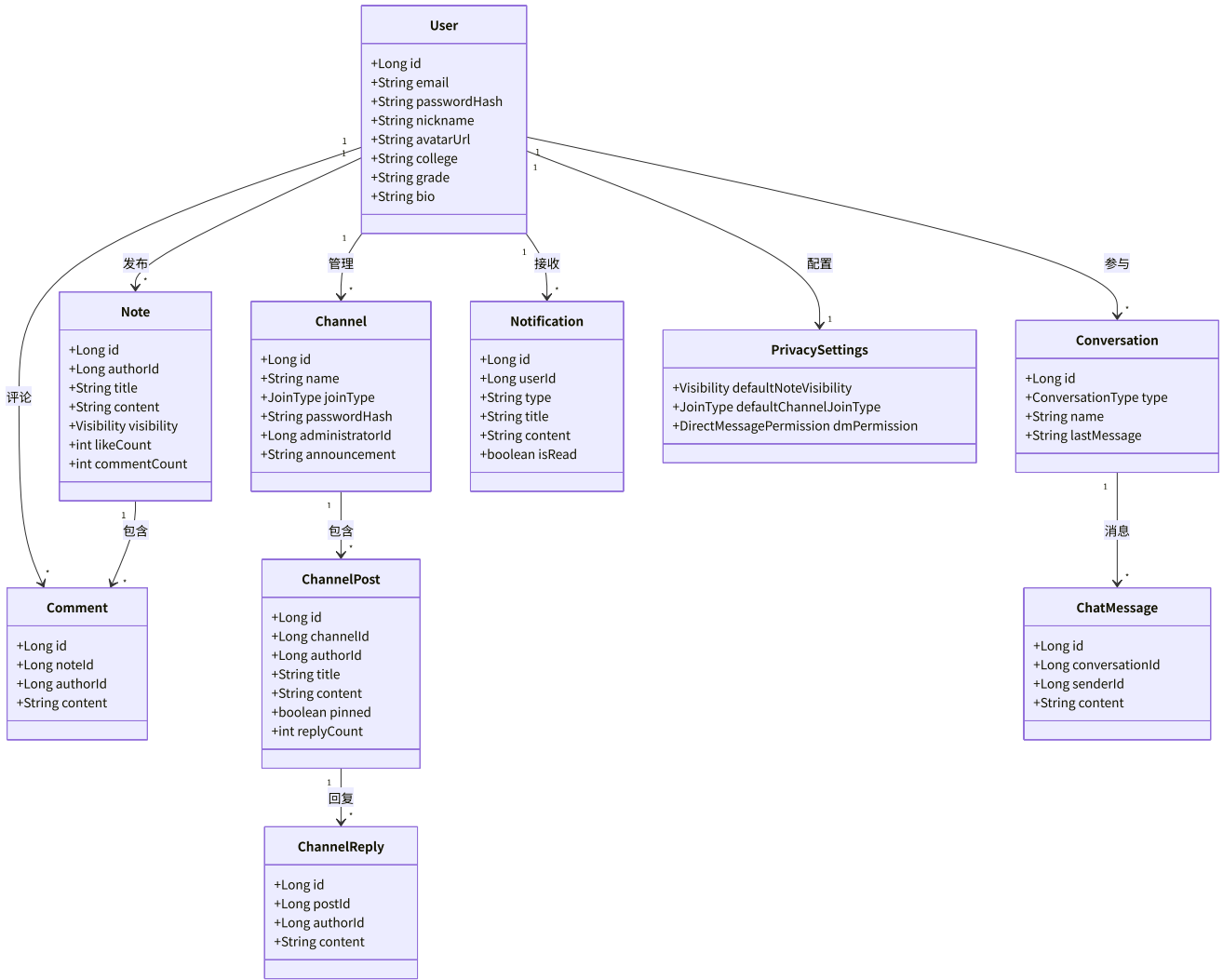




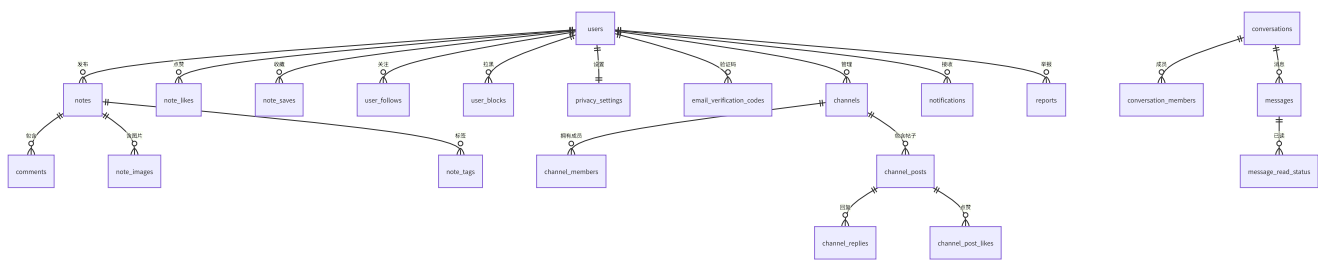
2.4 数据流图



2.5 类图



2.6 ER 图



2.7 数据字典

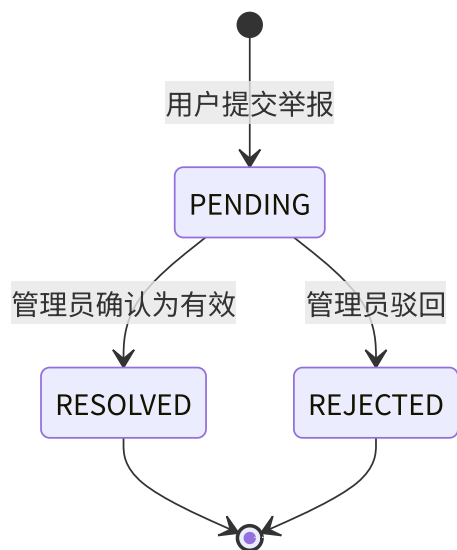
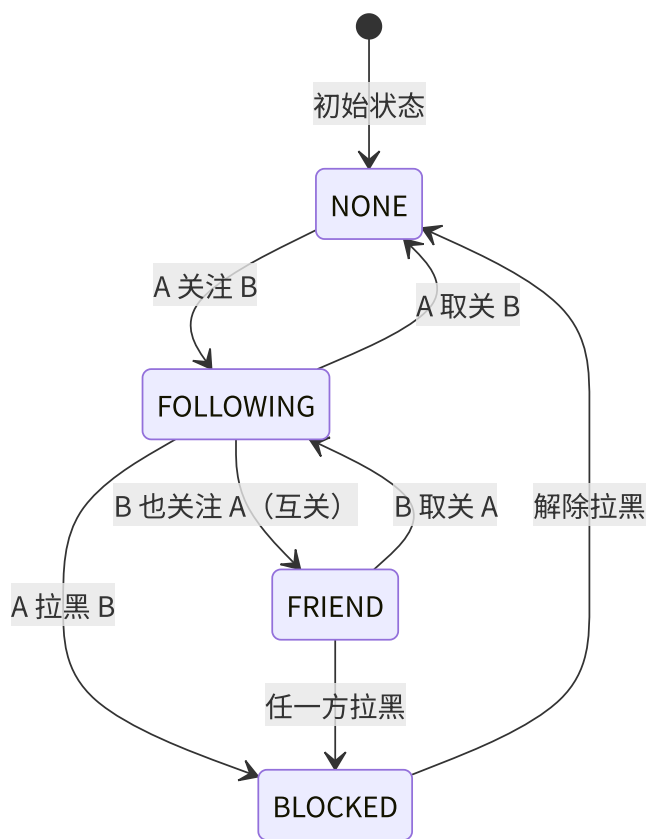
表名	关键字段	主键/约束
users	id, email(UNIQUE), password_hash, nickname, avatar_url, college, grade, bio	PK: id
email_verification_codes	id, email, scene, code_hash, expires_at, attempts, consumed_at	PK: id; INDEX(email,scene)
notes	id, author_id(FK), title(100), content(TEXT), visibility, like_count, comment_count	PK: id; INDEX(author_id,created_at)
note_images	id, note_id(FK), image_url(500), sort_order	PK: id; INDEX(note_id,sort_order)
note_tags	note_id(FK), tag(30)	PK: (note_id,tag); INDEX(tag)
comments	id, note_id(FK), author_id(FK), content(1000)	PK: id; INDEX(note_id,created_at)
note_likes	note_id(FK), user_id(FK)	PK: (note_id,user_id)
note_saves	note_id(FK), user_id(FK)	PK: (note_id,user_id)
user_follows	follower_id(FK), followed_id(FK), CHECK(前者≠后者)	PK: (follower,followed)
user_blocks	blocker_id(FK), blocked_id(FK), CHECK(前者≠后者)	PK: (blocker,blocked)
privacy_settings	user_id(PK+FK), default_note_visibility, default_channel_join_type, direct_message_permission	PK: user_id
channels	id, name(50), join_type(PUBLIC/PASSWORD), password_hash, administrator_id(FK), announcement(500), member_count	PK: id; INDEX(name)
channel_members	channel_id(FK), user_id(FK), role(ADMIN/MEMBER)	PK: (channel_id,user_id)
channel_posts	id, channel_id(FK), author_id(FK), title(100), content(TEXT), pinned, like_count, reply_count	PK: id; INDEX(channel_id,pinned,created_at)

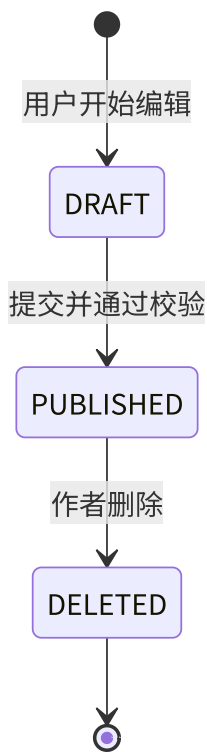
channel_replies	id, post_id(FK), author_id(FK), content(1000)	PK: id; INDEX(post_id,created_at)
channel_post_likes	post_id(FK), user_id(FK)	PK: (post_id,user_id)
conversations	id, type(PRIVATE/GROUP), name(50), last_message(2000), last_message_at	PK: id; INDEX(last_message_at)
conversation_members	conversation_id(FK), user_id(FK)	PK: (conversation_id,user_id)
messages	id, conversation_id(FK), sender_id(FK), content(2000), sent_at	PK: id; INDEX(conversation_id,sent_at)
message_read_status	message_id(FK), user_id(FK), read_at	PK: (message_id,user_id)
notifications	id, user_id(FK), type(40), title(100), content(500), target_id, is_read	PK: id; INDEX(user_id,is_read,created_at)
reports	id, reporter_id(FK), target_type, target_id, reason, status(PENDING/RESOLVED/REJECTED)	PK: id; INDEX(status,created_at)

枚举值：

枚举	取值	使用场景
Visibility	PUBLIC / FRIENDS / PRIVATE	notes.visibility、privacy_settings
JoinType	PUBLIC / PASSWORD	channels.join_type、privacy_settings
ConversationType	PRIVATE / GROUP	conversations.type
DirectMessagePermission	EVERYONE / FRIENDS_ONLY / NONE	privacy_settings.dm_permission
RelationStatus	NONE / FOLLOWING / FOLLOWER / FRIEND / BLOCKED	运行时计算（非持久字段）
ReportTargetType	NOTE / CHANNEL_POST / MESSAGE / USER	reports.target_type
ReportStatus	PENDING / RESOLVED / REJECTED	reports.status

2.8 状态图





三、非功能性需求说明

3.1 开发环境

层级	技术栈	版本
前端框架	React	19.2.0
构建工具	Vite	6.4.2
UI 图标	@phosphor-icons/react	2.1.10
后端框架	Spring Boot	3.5.16
Java 版本	JDK	17+
构建工具	Maven	3.9.x
API 文档	springdoc-openapi	2.8.17
数据库	MySQL	8.4 (Docker)
版本控制	Git + GitHub	

3.2 运行环境

服务	地址	启动命令
前端开发服务器	<code>http://127.0.0.1:5173</code>	<code>npm run dev</code>
后端 API	<code>http://127.0.0.1:8080/api/v1</code>	<code>mvn spring-boot:run</code>
Swagger UI	<code>http://127.0.0.1:8080/swagger-ui/index.html</code>	后端启动后即可访问
MySQL	<code>localhost:3306</code>	<code>docker compose up -d</code>

3.3 系统依赖

依赖	用途	备注
MySQL 8.4	业务数据持久化	Docker 镜像 mysql:8.4, utf8mb4 字符集
QQ 邮箱 SMTP	验证码发送	smtp.qq.com:465 SSL, 授权码存环境变量
Spring JDBC	数据库访问	双 Profile: mock/mysql
Spring Security Crypto	BCrypt 密码哈希	
当前版本不依赖: Redis、Nginx、消息队列、对象存储		

3.4 安全性

安全要求	实现方式
密码存储	BCrypt 哈希, 不可逆
接口鉴权	除 /auth/* 和 /health 外全量 Bearer Token 拦截
越权防护	用户仅可操作自己的笔记/评论; 频道管理员仅限己频道
输入校验	前端+后端双重校验 (长度、格式、枚举白名单)
SQL 注入	MyBatis 参数化查询
验证码安全	5 分钟过期、60s 冷却、5 次错误上限、哈希存储
敏感信息	数据库密码和 SMTP 授权码仅存环境变量, 不提交 GitHub

错误码体系:

code	HTTP	含义
40000	400	参数或邮箱格式错误
40001	400	邮箱或密码错误
40002	400	验证码错误或过期
40100	401	未登录或 Token 无效
40300	403	无权限或被拉黑
40301	403	非好友, 不可查看好友可见笔记
40302	403	未加入频道, 不可互动
40303	403	频道密码错误
40400	404	资源不存在
40900	409	状态冲突 (如邮箱已注册)
50000	500	服务端未处理异常

3.5 性能与可用性

类别	指标
性能	首页首屏 < 2s (本地环境); 列表分页 20 条/页, 最大 100 条
浏览器	兼容 Chrome / Edge 最新版
响应格式	统一 { code, message, data }; 字段小驼峰; 时间 ISO 8601

数据格式	请求/响应 JSON；ID 用数字
可维护性	后端 Controller→Service→Repository 分层；前端 src/api/ 按功能拆分
开发模式	双 Profile：mock（内存演示）/ mysql（持久化），通过 --spring.profiles.active 切换

四、拓展实现目标

- **活动组织与事件日历**：支持活动发布、报名、提醒和日历视图
- **平台管理员后台**：举报审核、违规处理、基础数据统计
- **推荐与发现优化**：基于标签和互动行为的内容排序
- **WebSocket 实时聊天**：将当前 HTTP 聊天升级为实时消息推送
- **文件存储服务**：图片外链替换为服务器存储或对象存储
- **统一搜索**：支持用户、笔记、频道、频道帖子的全局搜索
- **移动端适配**：优化手机浏览器下的布局和操作体验
- **站内智能助手**：接入 Agent，支持自然语言查询校园信息